ENTREGA FINAL DE PROYECTO — INGENIERÍA & DESARROLLO WEB

Leiti.app — Gestión Ganadera

Arquitectura React 18 + Programación Orientada a Objetos (POO) + Supabase PostgreSQL

INTEGRANTES DEL EQUIPO DE DESARROLLO:

DESARROLLADOR FULL-STACK & ARQUITECTURA REACT
Tomas Alejandro Garcia Benavides

DESARROLLADOR FULL-STACK & BASE DE DATOS / POO
Angel Daniel Cortes Garcia

1. Enlaces Oficiales del Proyecto (En Vivo y Repositorio)

Recurso / Plataforma	Enlace Directo (Clickeable)
Aplicación Web Publicada (GitHub Pages)	https://heladeriakreisel.github.io/leiti-app/
Acceso Directo SPA (app.html)	https://heladeriakreisel.github.io/leiti-app/app.html
Repositorio Oficial en GitHub	https://github.com/HeladeriaKreisel/leiti-app
Documentación Diagrama de Clases UML	https://github.com/HeladeriaKreisel/leiti-app/blob/main/DIAGRAMA_CLASES.md
Modelos POO en GitHub (src/models)	https://github.com/HeladeriaKreisel/leiti-app/tree/main/src/models
Base de Datos en la Nube (Supabase API)	https://cthvxtjwstyezwzuohdx.supabase.co (Proyecto: `leitti.appproyect`)

2. Evidencia de la Aplicación Funcionando y Conectada a Supabase

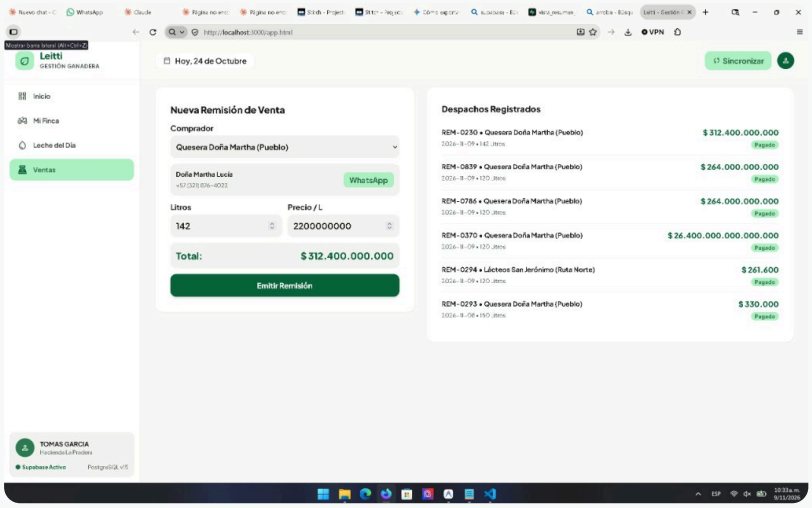


Figura 1: Interfaz de *Leiti.app* ejecutándose en navegador con conexión activa a Supabase PostgreSQL.

3. Estructura y Arquitectura de React + Diagrama de Clases UML

El sistema está estructurado bajo una arquitectura modular en capas (**Presentación / Vistas React → Servicios / API REST → Modelos de Dominio POO en ES6 → Persistencia PostgreSQL en Supabase**):

class Finca (src/models/Finca.js)

```
+ id: string
+ nombre_finca: string
+ propietario_nombre: string
+ municipio, departamento: string
+ capacidad_tanque_litros: number
+ temperatura_tanque: number
+ total_vacas, vacas_en_ordeno: number
+ meta_diaria_litros: number
```

```
+ calcularPorcentajeOcupacionTanque(litros): number
+ calcularCumplimientoMeta(litrosDia): number
+ calcularPromedioLitrosPorVaca(litrosDia): number
+ toDatabasePayload(): object
```

class RegistroLeche (src/models/RegistroLeche.js)

```
+ id: string
+ fecha: string | hora: string
+ turno: 'morning' | 'afternoon'
+ litros: number
+ precio_litro: number
+ flete_por_litro: number
+ tanque_temperatura: number
+ subtotal_cop, descuento_flete, neto_cop: number
```

```
+ calcularSubtotal(): number
+ calcularDescuentoFlete(): number
+ calcularNeto(): number
+ obtenerEtiquetaTurno(): string
+ toDatabasePayload(): object
```

class Cliente (src/models/Cliente.js)

```
+ id: string
+ clave_alias: string
+ nombre: string
+ empresa: string
+ contacto_nombre: string
+ telefono, telefono_wa: string
+ precio_acordado_litro: number
+ ruta: string
```

```
+ generarAlias(nombre): string
+ normalizarWhatsApp(telefono): string
+ obtenerEnlaceWhatsApp(mensaje): string
+ toDatabasePayload(): object
```

class VentaRemision (src/models/VentaRemision.js)

```
+ id: string
+ numero_remision: string
+ cliente_nombre: string
+ fecha, hora: string
+ litros: number
+ precio_litro: number
+ total_cop: number
+ metodo_pago, estado: string
```

```
+ generarNumeroRemision(): string
+ calcularTotal(): number
+ estaPagada(): boolean
+ alternarEstadoPago(): string
+ toDatabasePayload(): object
```

<<service>> SupabaseService (supabaseService.js)

```
- SUPABASE_URL: string
- SUPABASE_ANON_KEY: string
```

```
+ obtenerFinca(): Promise<Finca>
+ guardarFinca(fincaData): Promise<object>
+ listarRegistrosLeche(): Promise<RegistroLeche[]>
+ registrarOrdeno(registro): Promise<RegistroLeche>
+ listarClientes() / crearCliente() / eliminarCliente()
+ listarVentas() / emitirRemision() / actualizarEstadoVenta()
```

<<React Root>> App (src/App.jsx)

```
- state currentTab: string
- state finca: Finca
- state registros: RegistroLeche[]
- state clientes: Cliente[]
- state ventas: VentaRemision[]
```

```
+ useEffect(() ⇒ loadData(), [])
+ Renderiza: Sidebar, Header, DashboardView, MiFincaView, LecheDiaView, VentasView, ClientesView
```

Relaciones UML: 1) Composición: `Finca (1) *-- (0..*) RegistroLeche` (la finca origina los registros de ordeño diario). **2) Agregación:** `Cliente (1) o-- (0..*) VentaRemision` (un cliente recibe remisiones de venta pero existe independientemente). **3) Dependencia:** `App → SupabaseService → Modelos POO`.

4. Imágenes del Código Fuente del Proyecto

src/models/Finca.js & src/models/RegistroLeche.js — ES6 Classes / POO

```
// Clase de Dominio: Finca — Calcula ocupación del tanque, meta diaria y promedio por vaca
export class Finca {
  constructor(data = {}) {
    this.id = data.id || null;
    this.nombre_finca = data.nombre_finca || 'Hacienda La Pradera';
    this.propietario_nombre = data.propietario_nombre || 'TOMAS GARCIA';
    this.capacidad_tanque_litros = Number(data.capacidad_tanque_litros ?? 500);
    this.vacas_en_ordeno = Number(data.vacas_en_ordeno ?? 28);
    this.meta_diaria_litros = Number(data.meta_diaria_litros ?? 150);
  }
  calcularPorcentajeOcupacionTanque(litrosActuales) {
    if (!this.capacidad_tanque_litros || this.capacidad_tanque_litros <= 0) return 0;
    return Math.min(100, Math.round((Number(litrosActuales) / this.capacidad_tanque_litros) *
100));
  }
}

// Clase de Dominio: RegistroLeche — Liquidación financiera de cada ordeño
export class RegistroLeche {
  constructor(data = {}) {
    this.fecha = data.fecha || new Date().toISOString().split('T')[0];
    this.turno = data.turno || 'morning';
    this.litros = Number(data.litros ?? 0);
    this.precio_litro = Number(data.precio_litro ?? 2200);
    this.flete_por_litro = Number(data.flete_por_litro ?? 50);
  }
  calcularSubtotal() { return this.litros * this.precio_litro; }
  calcularDescuentoFlete() { return this.litros * this.flete_por_litro; }
  calcularNeto() { return this.calcularSubtotal() - this.calcularDescuentoFlete(); }
}
```

src/models/Cliente.js & src/models/VentaRemision.js — ES6 Classes / POO

```
export class Cliente {
  constructor(data = {}) {
    this.id = data.id || null;
    this.nombre = data.nombre || '';
    this.empresa = data.empresa || data.nombre || '';
    this.contacto_nombre = data.contacto_nombre || 'Encargado de Compras';
    this.telefono = data.telefono || '';
    this.precio_acordado_litro = Number(data.precio_acordado_litro ?? 2200);
    this.ruta = data.ruta || 'Ruta General';
  }
  obtenerEnlaceWhatsApp(mensaje = '') {
    const texto = encodeURIComponent(mensaje || `Hola ${this.contacto_nombre}, despacho
Leiti.app`);
    return `https://wa.me/${this.telefono_wa}?text=${texto}`;
  }
}

export class VentaRemision {
  constructor(data = {}) {
    this.numero_remision = data.numero_remision || `REM-${String(Math.floor(100 +
Math.random() * 900)).padStart(4, '0')}`;
    this.cliente_nombre = data.cliente_nombre || '';
    this.litros = Number(data.litros ?? 0);
    this.precio_litro = Number(data.precio_litro ?? 2200);
    this.estado = data.estado || 'paid';
  }
  calcularTotal() { return this.litros * this.precio_litro; }
  alternarEstadoPago() { return (this.estado = this.estado === 'paid' ? 'pending' : 'paid');
}
}
```

src/App.jsx — Componente Raíz, Estado Reactivo y Sincronización — React 18 Hooks

```
export default function App() {
  const [currentTab, setCurrentTab] = useState('inicio');
  const [finca, setFinca] = useState(null);
  const [registros, setRegistros] = useState([]);
  const [clientes, setClientes] = useState([]);
  const [ventas, setVentas] = useState([]);
  const [isSyncing, setIsSyncing] = useState(false);

  const loadData = async () => {
    setIsSyncing(true);
    const [fincaRes, registrosRes, clientesRes, ventasRes] = await Promise.all([
      getFincaData(), getRegistrosLeche(), getClientes(), getVentas()
    ]);
    if (fincaRes?.data) setFinca(fincaRes.data);
    if (registrosRes?.data) setRegistros(registrosRes.data);
    if (clientesRes?.data) setClientes(clientesRes.data);
    if (ventasRes?.data) setVentas(ventasRes.data);
    setIsSyncing(false);
  };

  useEffect(() => { loadData(); }, []);
  // Renderizado condicional de DashboardView, MiFincaView, LecheDiaView, VentasView y
ClientesView
}
```

app.html & src/services/supabaseService.js — Cliente REST PostgreSQL Supabase REST API

```
const SUPABASE_URL = "https://cthvxtjwstyezwzuohdx.supabase.co";
async function sbRest(path, options = {}) {
  const response = await fetch(`${SUPABASE_URL}/rest/v1/${path}`, {
    method: options.method || 'GET',
    headers: {
      'apikey': SUPABASE_ANON_KEY,
      'Authorization': `Bearer ${SUPABASE_ANON_KEY}`,
      'Content-Type': 'application/json',
      'Prefer': options.prefer || 'return=representation'
    },
    body: options.body ? JSON.stringify(options.body) : undefined
  });
  return await response.json();
}
```